Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Bearbeitungszeitraum: von 20. Januar 2026
bis 20. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: June 12, 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Aufgabensteller, Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in (Firma/Behörde/Hochschule):	OTH Amberg-Weiden
Betreuer in Firma/Behörde:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum: 20. Januar 2026	Abgabedatum: 20. Juni 2026

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

---

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Abstract:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Keywords: LaTeX, Textsatz, Formeln, Graphiken

Contents

Acronyms

BC Behavior Cloning

IL Imitation Learning

KM Knowledge Modality

LfO Learning from Observation

LfV Learning from Videos

MDP Markov decision process

ML Machine Learning

RL Reinforcement Learning

Symbole, Formelzeichen und Einheiten

Chapter 1

Foundations

This chapter provides an overview of the key concepts and theories that form the foundation of the following chapters.

1.1 Introduction and Mathematical Notation of Formulas

Capital Letters are used for random variables, e.g. S represents the set of all states in which the robot can be and A represents the set of all possible actions it can perform.

Lowercase Letters are used for specific values of these random variables, e.g. s stands for a specific robot state and a for a specific action that the robot performs. A lowercase letter is usually followed by a time index $_t$, representing the specific action taken by the robot at time step t. In some cases the time index is omitted for simplicity, e.g. a instead of a_t and the next state or action is then denoted as s' or a' instead of s_{t+1} or a_{t+1}.

Capital Letters in Bold represents Matrices, e.g. the matrix $\mathbf{X}$.

Lowercase Letters in Bold are used for specific values of (random) variables that are vectors, e.g. $\mathbf{x}$ represents a specific vector.

1.2 Glossary

Robot Trajectory:

A robot trajectory describes how the robot moves from a starting state to a target state, including position, orientation and, if applicable, speeds, accelerations, and forces acting on the robot. A robot trajectory is a temporal sequence of actions and the resulting robot states.

Inference:

Inference refers to the process of using a trained model to make predictions or decisions based on new, unseen data. In the context of robotics, inference involves the robot applying its learned policy in real world or simulation to execute a given task.

Annotation:

Annotating refers to the process of labeling data, in which data is enriched with additional information. For example, this may include assigning class labels, marking target objects in images, or labeling states and actions within robot trajectories. Annotated, or labeled, data forms the foundation for supervised learning approaches.

Weakly Supervised Learning:

Weakly supervised learning is a Machine Learning (ML) approach that utilizes limited or incomplete labeled data for training. In this context, the model learns from a combination of labeled and unlabeled data, or from data with noisy or imprecise labels. This approach is particularly relevant in robotics, where obtaining fully annotated data can be costly and time-consuming. Weakly supervised learning allows models to leverage large amounts of unlabeled data while still benefiting from the guidance provided by the available labeled data.

Self-Supervised Learning:

Self-Supervised Learning is a ML approach where the model learns to predict parts of the input data from other parts of the same data. This method does not require external labels, as the model generates its own supervisory signal from the data itself. For example, Large Language Models (LLMs) learn to predict the next word or masked words in a sentence based on the surrounding context, where the correct target words are already contained within the training data. Similarly, in computer vision, a model may learn to reconstruct masked regions of an image, with the original image providing the target information.

1.3 Machine Learning

1.3.1 Core Machine Learning Categories

There are various approaches to training an AI model for a robot. These involve methods from the four core ML categories, which can also be combined.

Supervised Learning:

In supervised learning, a model is trained using labeled training data. This means that, for the input data, the output data to be predicted are known. The goal is to train a model that correctly predicts the output data and makes reliable predictions on unseen data.

Unsupervised Learning:

In contrast to supervised learning, unsupervised learning uses unlabeled data for model training. The model tries to recognize structures, patterns or relationships in the data on its own.

Semi-Supervised Learning:

Semi-supervised learning is a combination of supervised and unsupervised learning. In this approach, a model is trained using a small amount of labeled data and a large amount of unlabeled data. The goal is to leverage the advantages of both approaches, particularly when annotating data is time-consuming or cost intensive. This is relevant in robotics, as the data used in that field is often difficult to label.

Reinforcement Learning:

Reinforcement Learning (RL) differs fundamentally from the previously mentioned approaches. Instead of learning from fixed datasets, an agent actively interacts with its environment and tries to find out an optimal strategy to reach a defined goal. Through its actions, it influences the state of the environment and subsequently receives feedback in the form of rewards or penalties, see Figure 1.1 for a short overview. In the context

of robotics, the robot is the agent. The objective is to learn a strategy (= policy) that maximizes the cumulative reward over time for the given task.

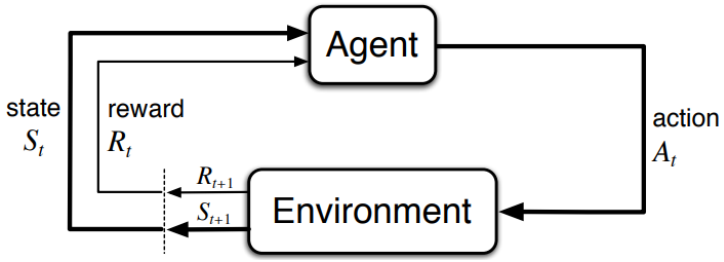


Figure 1.1: Base concept of Reinforcement Learning. Image source: [1].

The agent proceeds in its environment according to the trial-and-error principle to learn this optimal policy. Safety is a critical concern during training, as the agent may execute unpredictable actions while exploring the state space. For this reason, simulation environments are a central component of training and are frequently used for pre-training the policy. The policy is then subsequently often fine-tuned in the real-world environment. Because the agent learns through trial and error, more efficient, unknown, and creative solutions can be found that a human might not have come up with. See section 1.6 for more details.

1.4 Deep Learning

1.4.1 Neurons

1.4.2 Neural Networks

Activation Functions

Training Neural Networks

1.4.3 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep neural network architecture designed to process grid-like data, especially images. CNNs are particularly effective for visual tasks because they exploit the spatial structure of an image and learn local patterns such as edges, corners, and textures. Unlike fully connected networks, CNNs do not treat every pixel independently. Instead, they use small learnable filters that are applied across the image to detect relevant structures at different positions.

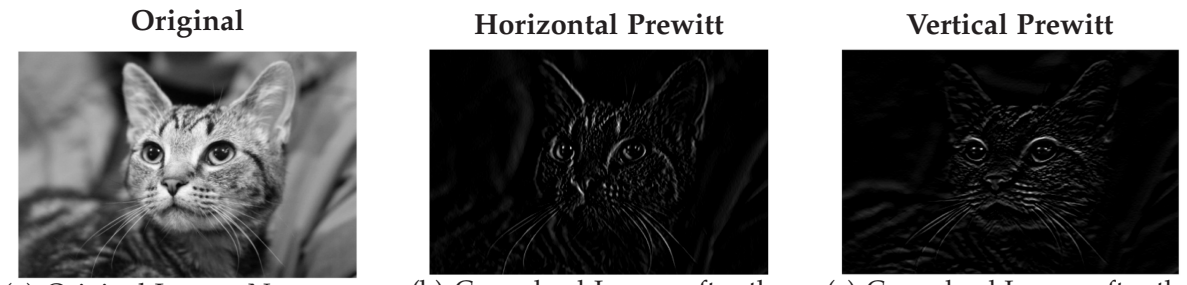
Convolution

The main idea of a CNN is the convolution operation, where a filter, also known as kernel, is applied to the image to extract features. For example, the Prewitt kernels, also known as edge detection filters, can be used to detect horizontal and vertical edges

in an image:

$$K_{\text{horizontal}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad K_{\text{vertical}} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \tag{1.1}$$

Figure 1.2 shows the result, after applying the Prewitt kernels to a grey-scale image. There exists a wide variety of predefined filters to extract various of visual structures from the image.



(a) Original Image: No convolution is applied

(b) Convolved Image, after the horizontal Prewitt kernel is applied to the original image

(c) Convolved Image, after the vertical Prewitt kernel is applied to the original image

Figure 1.2: Image-based instructions for task execution. Image source: [**xxx**].

A Filter or kernel is defined as a small matrix of weights, is moved across the image and element-wise multiplied with the underlying pixel values. This is possible, because an image is technically a matrix of pixel values too. The resulting values are stored in a new matrix called a feature map, which highlights the patterns detected by the kernel. Since the same kernel is reused at every pixel location, CNNs are able to detect the same feature independent of its position in the image. Figure Figure 1.3 illustrates the mathematical convolution operation, where a 3×3 kernel is applied to a 6×6 image. The results are stored in a 3×3 matrix.

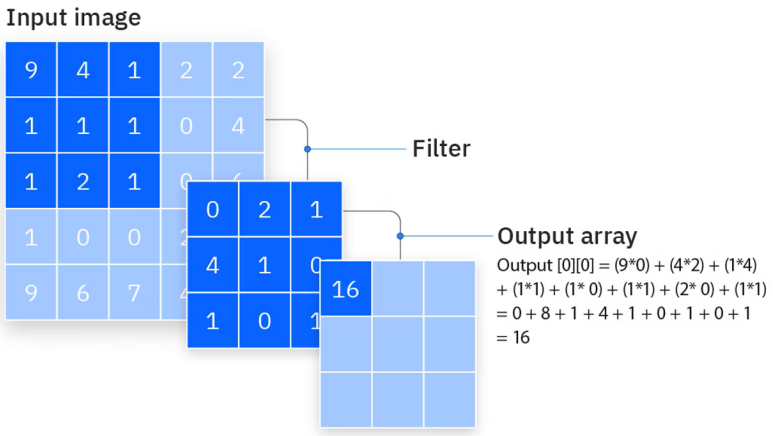


Figure 1.3: CNN Convolution Introduction. Image based on [**xxx**].

Figure Figure 1.4 shows how the kernel (dark blue) is slided across the image (light blue) step by step. At each location the element-wise products are summed up to obtain one output value, which is stored in a feature map (olive green).

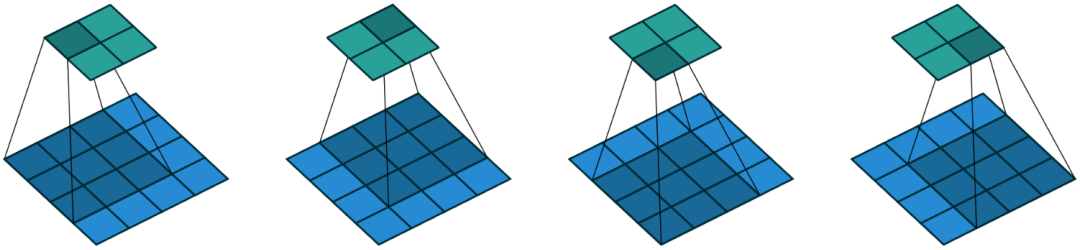


Figure 1.4: CNN Convolution Step-by-Step. Image based on [**xxx**].

Figures Figure 1.5 shows an alternative view of the convolution process.

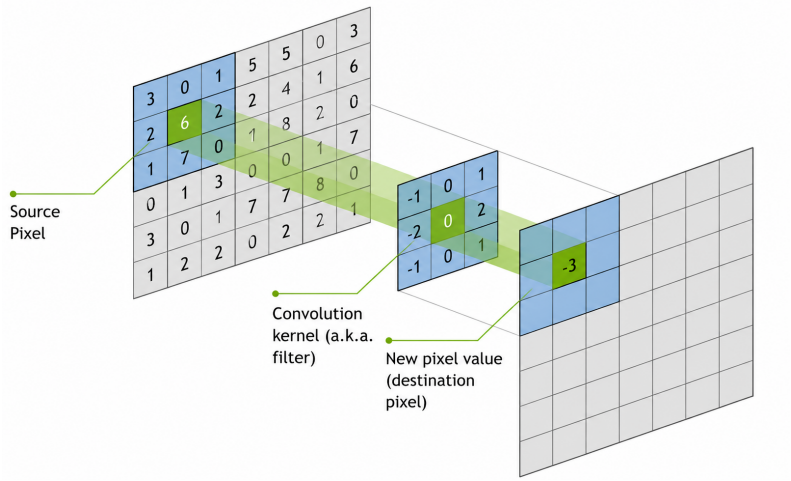


Figure 1.5: CNN Convolution Introduction. Image based on [**xxx**].

Padding

As seen in the Figures 1.3, 1.4 and 1.5, the feature map is smaller than the original image, because of the convolution operation. To preserve the spatial resolution of the input image, padding is often added around the border before applying the convolution, see Figure 1.6. With zero-padding, the image size can remain unchanged after the convolution, which is useful when multiple convolution layers are stacked.

A CNN consist of multiple kernels, which are applied in parallel and in multiple layers to the image and the resulting feature maps. Importantly, in a CNN the kernels are not fixed manually, instead they are learned during training, allowing the network to find the optimal filters for the task success.

Activation Functions

After the convolution operation, a non-linear activation function is applied to the resulting feature map. Activation functions introduce non-linearity into the network,

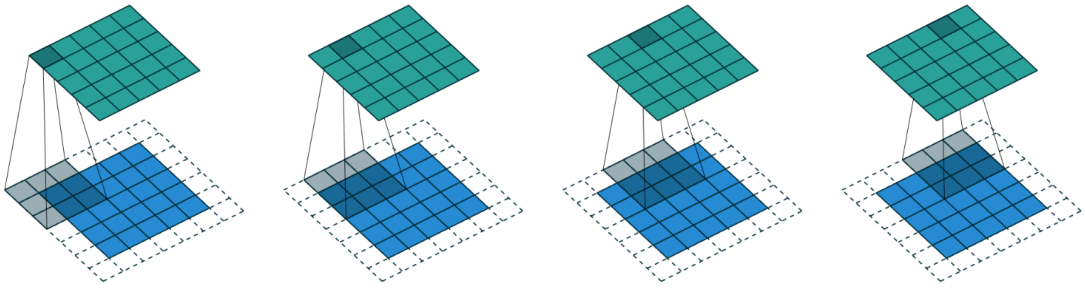


Figure 1.6: CNN Convolution with Padding. Image based on [**xxx**].

allowing CNNs to learn complex visual patterns that cannot be represented by linear transformations alone.

The most commonly used activation function is the Rectified Linear Unit (ReLU): which replaces all negative values with zero while leaving positive values unchanged:

$$f(x) = \max(0, x), \tag{1.2}$$

By applying ReLU after each convolution, the network can learn increasingly complex feature representations across multiple layers.

Pooling

Another central component of CNNs is the pooling layer. Pooling reduces the spatial size of the feature maps while retaining the most important information. This lowers the computational cost and increases robustness to small shifts in the input image.

A common choice is a 2×2 pooling window with a stride of 2. In this case, the feature map is divided into non-overlapping 2×2 regions, and each region is summarized by a single value. As a result, the width and height of the feature map are reduced by a factor of two.

Two common pooling methods are max pooling and average pooling, as illustrated in Figure 1.7. In max pooling, the largest value within each pooling region is selected and propagated to the output feature map. In average pooling, the mean value of all elements within the region is computed. Both methods reduce the spatial resolution while preserving the most relevant information contained in the feature map.

Pooling layers are typically applied after convolution layers and allow the network to build increasingly compact feature representations. While early convolution layers extract local patterns such as edges and textures, pooling helps summarize these features and enables deeper layers to focus on higher-level visual structures.

CNN Pipeline

The common CNN processing sequence is: (1) convolution, (2) activation, and (3) pooling, while the activation function is often not mentioned explicitly. A CNN usually consists of several repeated blocks of convolution and pooling layers. Early layers

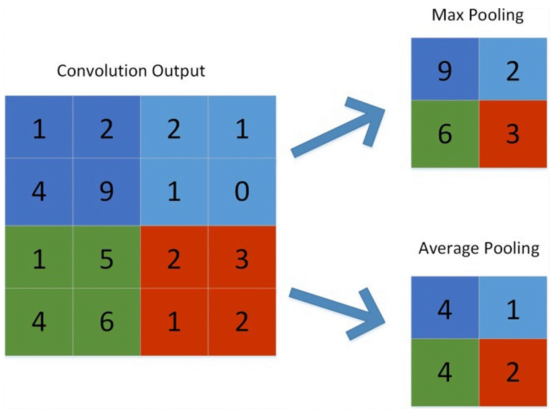


Figure 1.7: CNN Pooling. Image based on [**xxx**].

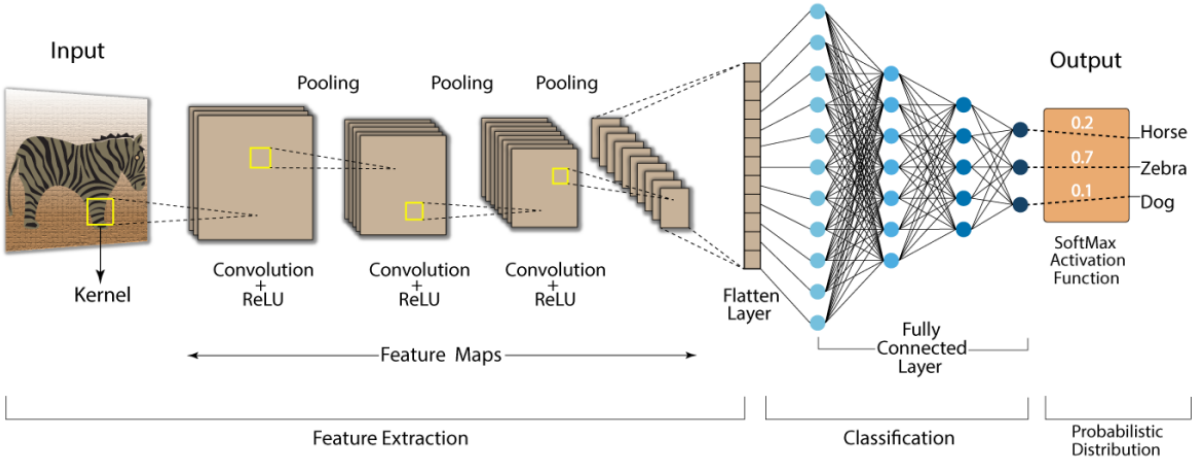


Figure 1.8: CNN Pipeline. Image based on [**xxx**].

detect simple features such as edges and corners, while deeper layers combine these basic features into more complex structures. After several convolution and pooling stages, the feature maps are flattened into one big vector and passed through a MLP or a classification head in a classification task to produce the final predictions, see Figure 1.8. Through backprobagating the prediction error, the networks parameter are adjusted during training.

RGB Images

The same principle can be extended to RGB images. While a grayscale image contains only a single channel, an RGB image consists of three color channels representing red, green, and blue values. Consequently, a convolution kernel must span all input channels. For a kernel size of 3×3, a common way is to use an $3 \times 3 \times 3$ kernel for an RGB image. This kernel is applied across all three color channels simultaneously, and the resulting values are summed up to produce a single output value in the feature map. By learning appropriate kernels during training, CNNs can effectively capture complex visual patterns across all color channels in RGB images.

1.4.4 Transformer

A Transformer is a deep neural network architecture designed to process sequential data, especially text. It's the foundation for most modern LLMs, enabling them to process and understand language effectively. Unlike earlier architectures, which process tokens one after another, Transformers compare all tokens in a sequence in parallel and learn which parts of the input are relevant to each other. This is done through the attention mechanism and was the core innovation of the Transformer architecture. The main idea is simple: when the model reads one token, it does not treat all other tokens equally. It assigns higher importance to the tokens that are more relevant in the current context. This makes Transformers especially powerful for language tasks, because meaning often depends on long-range dependencies in the sentence.

Architecture

The original Transformer architecture was introduced by Vaswani et al. (2017) and consists of two parts: the encoder and the decoder. See Figure 1.9 for an overview.

1. **Encoder:** The encoder reads the full input sequence and converts it into a contextual latent representation. Each token is transformed not only based on its own identity, but also based on the other tokens in the sequence. In this way, the encoder builds a rich internal representation of the input.
2. **Decoder:** The decoder generates the output sequence autoregressively, meaning one token at a time. To produce the next token in the output sequence, it uses at each prediction step: (1) The contextual representation from the encoder; (2) The previously generated output tokens from the decoder. This allows the decoder to produce the next token while taking the source sequence into account.

While the examples in this section are based on text processing to facilitate understanding of the Transformer architecture, the underlying principles generalize to other modalities, such as images or videos.

1. Input Embeddings

Before a Transformer can process data, the data must be converted into a numerical representation. This is the role of the input embedding layer. Given an input text such as "Data visualization empowers users to", the model first tokenizes the text, then maps each token to a vector representation, and finally adds positional information. The resulting embedding combines semantic meaning with information about token order.

1. **Tokenization:** The input text is split into smaller units called tokens. These tokens can be a word, a subword or a single character. For example, the sentence might be tokenized into ["Data", "visualization", "empowers", "users", "to"]. Each token is assigned a unique token ID.

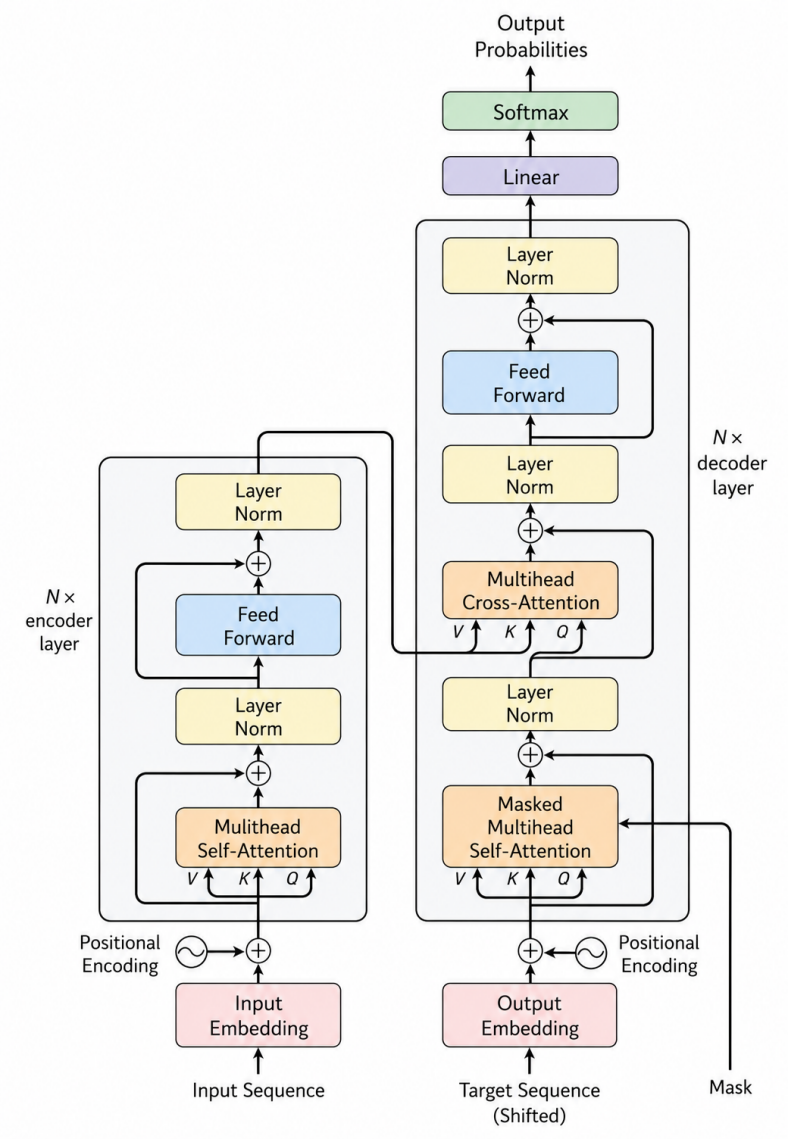


Figure 1.9: Transformer Architecture. Image based on [2].

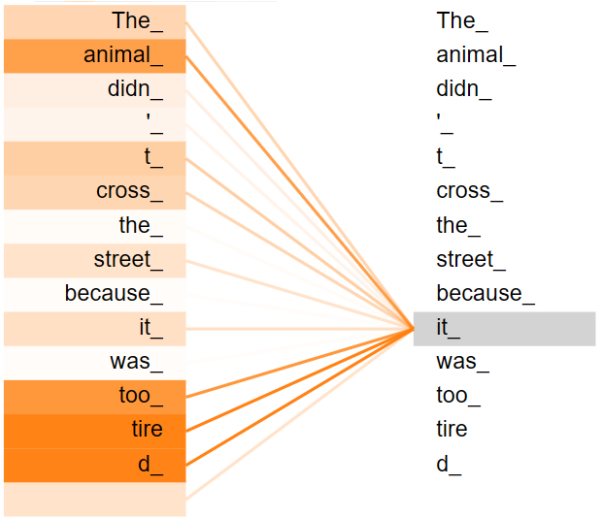


Figure 1.10: Attention Weights of the word “it” in a sentence. Image source

2. **Embedding:** Each token is then mapped one-to-one to a high-dimensional vector representation. This embedding captures the semantic meaning of the token. For example, the token "Data" might be represented as a 768-dimensional vector in the embedding space.
3. **Positional Encoding:** Since the Transformer does not process tokens sequentially, it needs an explicit way to represent token order. Positional encodings provide this information. Depending on the model, these encodings may be fixed or learned. They are added to the token embeddings so that the model can distinguish between, for example, “dog bites man” and “man bites dog”.
4. **Final Embedding:** The final input representation is obtained by combining token embeddings and positional information, typically through element-wise addition. This representation is then passed into the Transformer layers.

2. Attention Mechanism

The Attention mechanism is responsible for capturing contextual relationships between tokens by directly computing their relevance to one another. This allows the model to understand long-range dependencies and contextual relationships of the tokens, leading to more coherent and relevant output. In Figure 1.10 the attention weights of the word “it” in the sentence “The animal didn’t cross the street because it was too tired” are visualized, after the input is processed through the Encoder attention mechanism. The attention weights allows the model to decide which other tokens in the sequence are most relevant.

Compute Attention Weights. In the self-attention mechanism, each token embedding vector $\mathbf{x}^{(i)}$ is transformed into three different representations: query $\mathbf{q}^{(i)}$, key $\mathbf{k}^{(i)}$ and value $\mathbf{v}^{(i)}$. These representations are obtained through learnable linear transformations with parameters (W_q, W_k, W_v) that are optimized during training.

- **Query $\mathbf{q}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_q$:** The query vector represents the token for which we want to compute attention. It is used to determine how much attention this token should pay to other tokens in the sequence.
- **Key $\mathbf{k}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_k$:** The key vector represents the token against which the query is compared. It is used to compute the relevance score between the query and the token.
- **Value $\mathbf{v}^{(i)} = \mathbf{x}^{(i)}\mathbf{W}_v$:** The value vector contains the actual information of the token. It is weighted by the attention scores to produce the final output of the self-attention mechanism.

Self-Attention. The query vectors are then compared with the key vectors of all tokens in the sequence using the dot product (scalar product), producing relevance scores that indicate how strongly tokens are related to one another. To stabilize training, these scores are scaled by the square root of the key dimension d_k and normalized using a softmax function, resulting in attention weights. Finally, the attention weights are applied to the corresponding value vectors $\mathbf{v}^{(i)}$ and these weighted values are aggregated to produce the new context vector $\mathbf{z}^{(i)}$ and the output of the self-attention layer, see Figure 1.12.

$$\mathbf{z}^{(i)} = \text{softmax}\left(\frac{\mathbf{q}^{(i)} \cdot \mathbf{k}^{(i)T}}{\sqrt{d_k}}\right) \cdot \mathbf{v}^{(i)} \tag{1.3}$$

In this way, each token can incorporate information from the most relevant tokens in the sequence, enabling the model to capture contextual dependencies. But What does this mean technically? The value vector $\mathbf{v}^{(i)}$ represent the token's informatation in an semantic learned high-dimensional embedding space. Through the attention machanism changes the vector it's direction and magnitude in this embedding space and point to another place, resulting in a other semantic meaning. In Figure 1.11 is shown how the vector of the word "creature" is shifted in the embedding space based on the context "a fluffy blue creature". This illustrates how the attention mechanism allows the model to dynamically adjust the meaning of tokens based on their relationships with other tokens in the sequence.

Instead of computing attention separately for each token, the Transformer performs the computation for the entire sequence using matrix operations, see Figure 1.13. The matrix product $\mathbf{QK}^T$ computes the pairwise similarity scores between all tokens in the sequence in parallel, resulting in the attention score matrix.

$$\mathbf{Z} = \mathit{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{1.4}$$

Multi-Head Attention. While a single attention mechanism can capture relationships between tokens, different types of dependencies may exist simultaneously within a sequence. To address this, the Transformer employs multi-head attention, which consists of several attention heads operating in parallel. Each head learns its own query, key, and value projections and therefore focuses on different aspects of the input

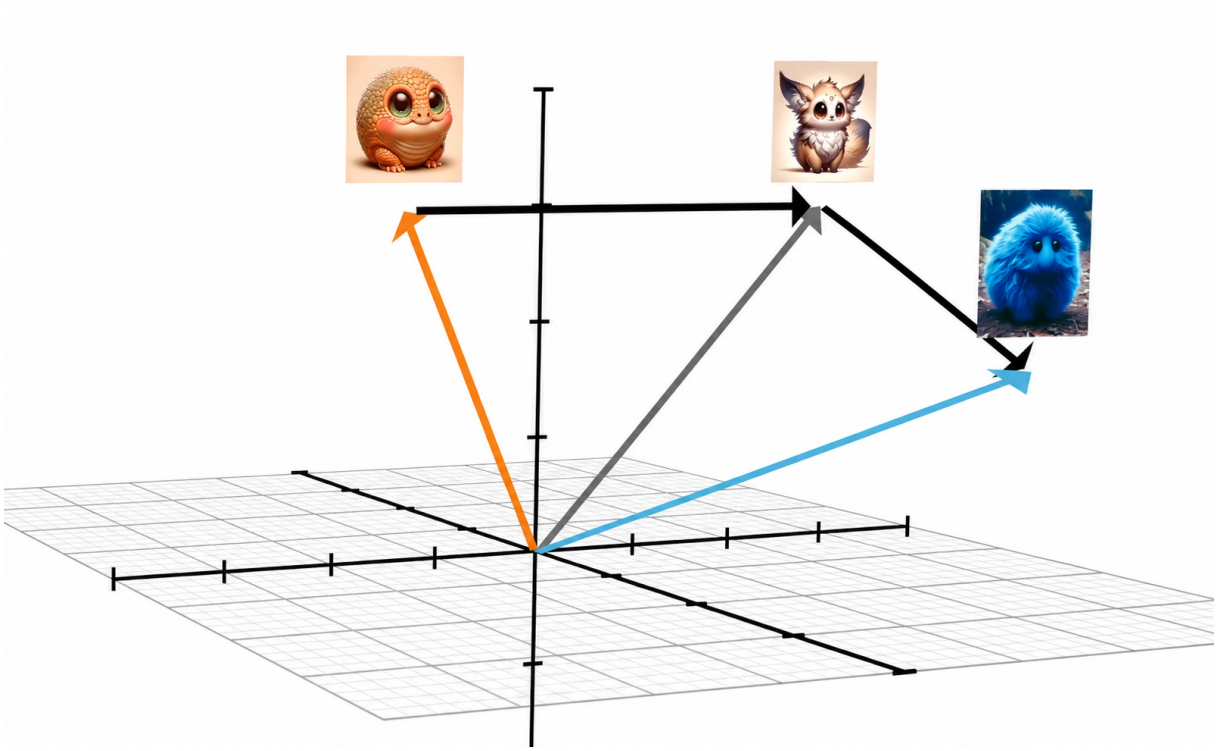


Figure 1.11: Embedding Space of the Value Vectors: The Vector of "creature" is shifted in the Embedding space, based on the context.

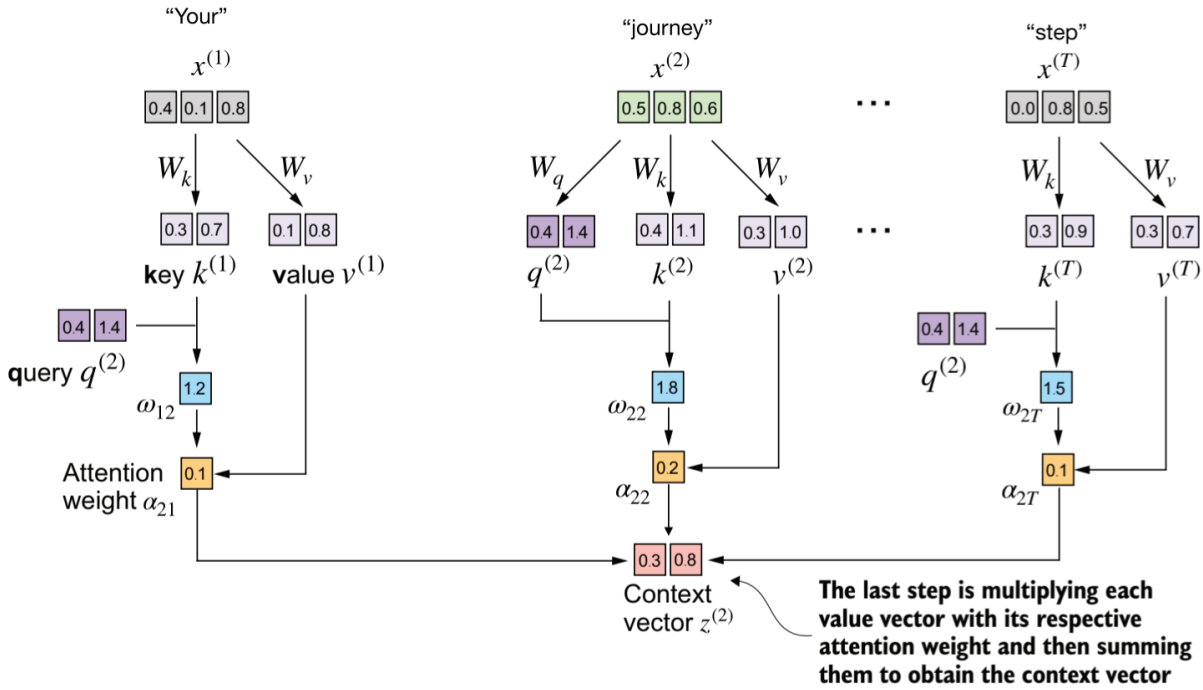


Figure 1.12: Self-attention: Compute the new context vector of each input embedding vector by combining the (attention)-weighted value vectors. Image source [xxx].

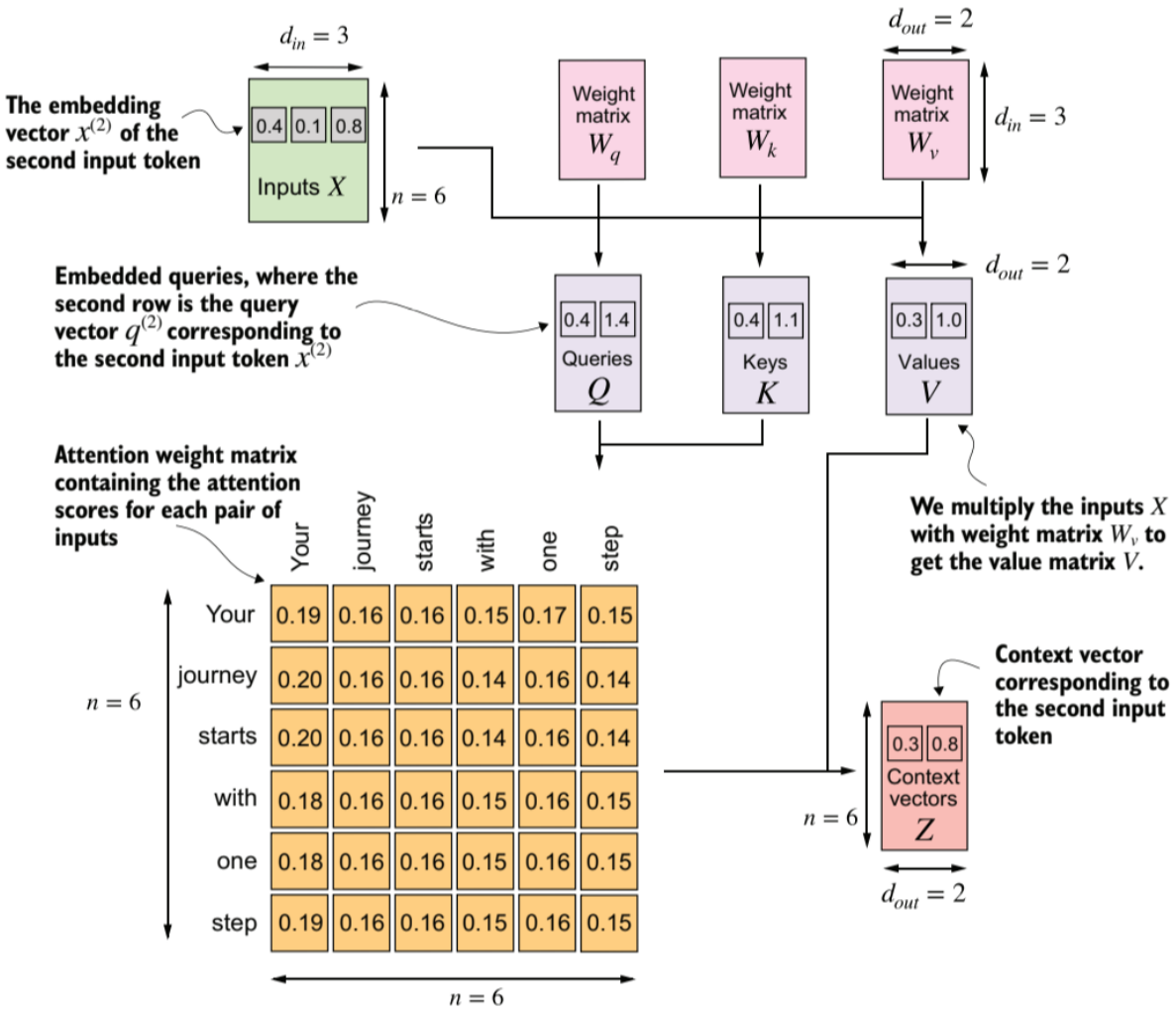


Figure 1.13: Single self-attention Block. Image source [**xxx**].

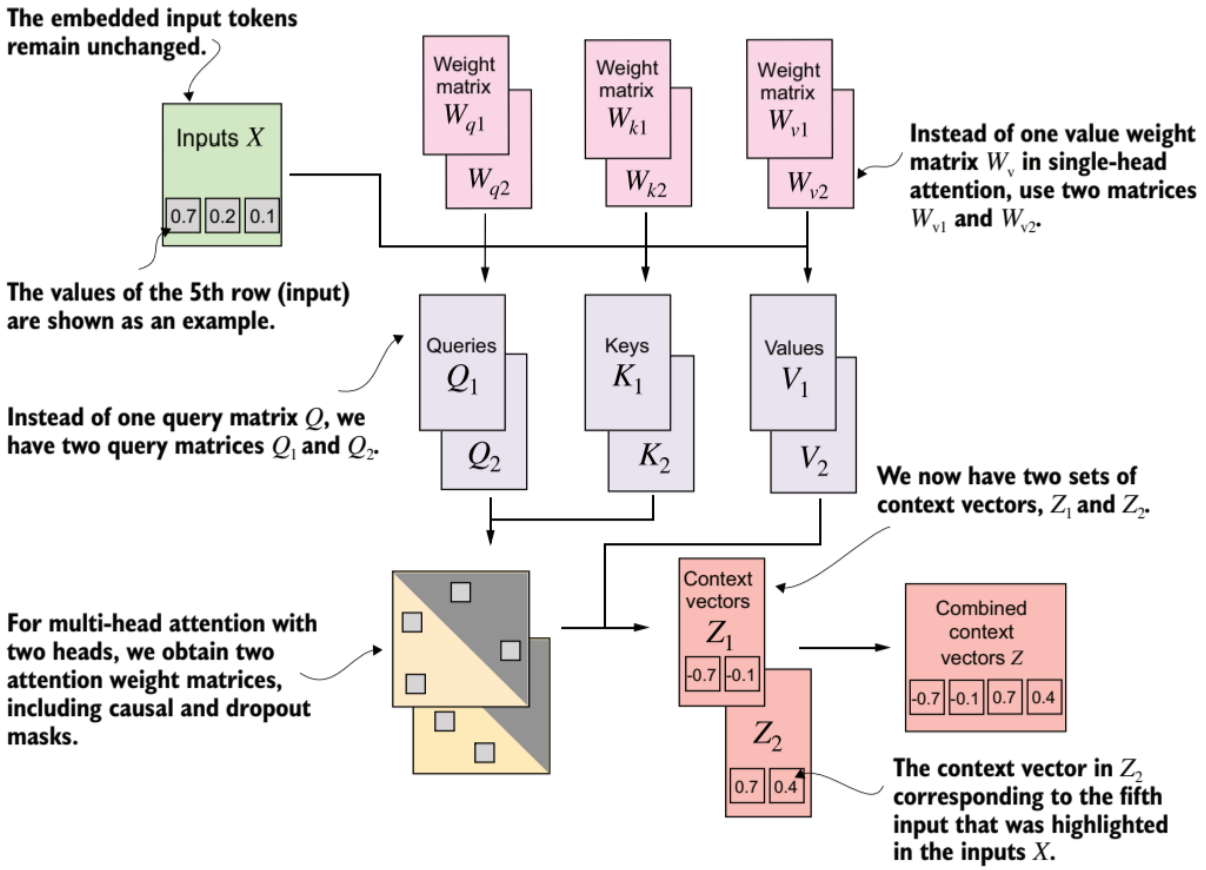


Figure 1.14: Multi-Head Attention Block. Image source [**xxx**].

data. For example, one head may capture syntactic relationships, while another focuses on semantic dependencies. The outputs of all heads are concatenated and projected through a linear layer to form the final representation, see Figure 1.14. This increases the model's ability to learn diverse linguistic patterns and contextual relationships. The number of heads is chosen such that the input embedding dimension is split across the heads. For example, if the input embedding dimension is 768 and there are 12 heads, each head would operate on a subspace of 64 dimensions. Otherwise, the output dimensions will not match the input dimensions. Technically the Query, Key and Value matrices of each head are combined computed within a larger matrix multiplication and then splitted up into the individual parts for each head, that every head can produce its own attention matrix.

3) Encoder Block:

After the self-attention computation, the resulting representations are passed through a layer normalization step, followed by a feed-forward network (MLP). A second layer normalization is applied after this transformation. This sequence of operations forms a single Transformer block, which can be stacked multiple times to increase model capacity.

The MLP (Multi-Layer Perceptron) operates independently on each token represen-

tation. In contrast to the attention mechanism, which is responsible for exchanging information between tokens, the MLP refines and transforms each token embedding individually. It typically consists of two linear transformations with a non-linear activation function in between, allowing the model to learn more complex feature transformations.

A key component of the Transformer architecture is the use of residual connections, which are applied around both the attention block and the feed forward network. These are shortcut connections that add the input of a sub-layer directly to its output. This mechanism improves gradient flow during backpropagation and mitigates the vanishing gradient problem, which is especially important in deep networks composed of many stacked Transformer blocks.

The output of the final encoder layer is a sequence of contextualized vectors, where each vector corresponds to one input token enriched with information from the entire sequence. This representation can then be used for downstream tasks such as classification, regression, or sequence generation.

4) Decoder Block:

The decoder receives an input sequence that is first transformed using the same embedding procedure as the encoder (token embeddings + positional information). During generation, this input typically consists of previously generated output tokens. The embedded sequence is then processed through two distinct attention mechanisms: masked multi-head self-attention and cross-attention.

Masked Multi-Head Attention. Masked multi-head self-attention operates on the decoder's own input sequence, but with a causal mask that prevents each position from attending to any future positions. The causal mask ensures that future tokens receive an attention weight of zero, making them invisible to the current position and ensures that the prediction for the next token depends only on previously generated tokens.

Cross-Attention. In cross-attention, the decoder's hidden states from the masked attention layer serve as queries Q, while the encoder's output serve as keys K and values V, allowing the decoder to condition its predictions on the encoded source sequence.

After these attention layers, the resulting representations are passed through a linear projection head followed by a softmax function, producing a probability distribution over the set of possible output elements. The element with the highest probability is selected and appended to the generated sequence. This process repeats autoregressively until a special end-of-sequence symbol is produced. In our text example, the vocabulary token with the highest probability is selected and appended to the generated sequence.

1.4.5 Vision Transformer

The Vision Transformer (ViT) [3] extends the Transformer architecture, originally developed for natural language processing, to computer vision tasks by applying

the self-attention mechanism to image patches instead of words. In computer vision, attention enables a model to focus on relevant regions of an image and to capture and understanding relationships between different areas of an image. This allows the model to identify important objects and effectively model spatial dependencies across the entire image.

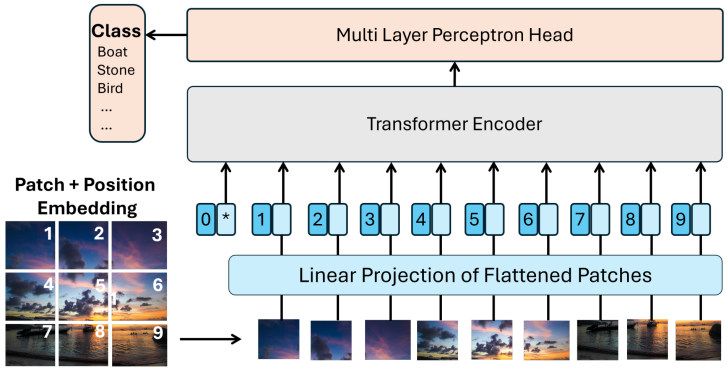


Figure 1.15: Vision Transformer: Overview of the architecture. Image source [4].

To process an image, ViT first divides the input image into a sequence of fixed-size, non-overlapping patches, see Figure 1.15. Each patch is flattened and linearly projected into a fixed-dimensional embedding vector, analogous to token embeddings in natural language processing.

Since the Transformer architecture does not inherently encode spatial information between the image patches, learnable positional embeddings are added to the patch embeddings. In addition, a learnable classification token (CLS token), denoted by an asterisk (*) in Figure 1.15, is prepended to the sequence and serves as a global representation of the whole image. During the attention computation, the CLS token interacts with all patch embeddings and gradually aggregates information from the entire image.

The resulting sequence is processed by a standard Transformer encoder consisting of multi-head self-attention and feed-forward layers. Through self-attention, the model learns global relationships between image patches and can capture long-range spatial dependencies that are difficult to model with traditional CNN approaches.

After the final encoder layer, the output representation of the CLS token serves as a global representation of the image and is passed through a multi-layer perceptron (MLP) head to generate the final prediction, such as class probabilities for image classification tasks.

Vision Transformers have demonstrated competitive performance on large-scale image classification benchmarks such as ImageNet [5], highlighting the effectiveness of attention-based architectures for visual representation learning [3]. In imitation learning, Vision Transformers can be used as visual encoders that extract global scene representations from camera observations, which are subsequently used by the policy network to predict actions.

1.5 Foundation Model vs. Multi-Model Model

1.6 Reinforcement Learning

RL is both a fundamental discipline of ML and a distinct approach within Robot Learning. The robot learns a policy through trial and error using rewards provided from the given task and the environment. The actions performed by the agent are evaluated with a reward score, and this score is then used to gradually optimize the policy. See also figure 1.1 for a general graphical overview. Below are the key terms in RL explained in relation to robotics:

- **Agent:** The agent is the entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task.
- **Environment:** The environment is the setting in which the agent/the robot operates. In other words, the environment is the robot's workplace, where it is deployed. In addition to the robot itself, it includes all relevant objects with which the robot interacts. The environment can be either the real world or a simulated environment in which the robot interacts during training or inference.
- **Robot Actions:** The Robot actions are simply the control commands the robot can send to its actuators, e.g. open or close the gripper, move the arm to position X, Y, Z, or the joint angles of the individual robot axes, etc.
- **Reward:** The reward is a numerical value that indicates how well the robot is performing in relation to the task objective. For example, if the robot successfully picks up an object, it might receive a positive reward, while if it fails or drops the object, it might receive a negative reward. The goal of Reinforcement Learning is to learn a policy that maximizes the cumulative reward over time.
- **Robot State:** The State is a complete description of the situation in the environment at a particular point in time. The robot state represents the current situation of the robot and its environment. It can include various types of information, such as the robot's position, orientation, joint angles, sensor readings, and even visual input from cameras [6]. The state is crucial for the robot to make informed decisions about which actions to take.
- **Observation:** An observation is the information that the robot receives from its sensors at a given time step. It may not provide a complete picture of the environment, and thus may not fully represent the true robot state. In our case, the observation is mostly equivalent to the state [6].
- **Policy:** The policy can be interpreted as the brain of the agent. The policy determines which action should be taken next when the agent is in a particular state in order to achieve the task objective. The Policy controls the robot's motion and actions.

Formally, a policy can be represented as

$$\pi_\theta(a \mid s) \tag{1.5}$$

which defines a probability distribution over actions a, conditioned on the current state s. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. In the classical meaning, the action a corresponds for example to a robot command, such as moving the gripper to a target position (x, y, z) or setting a joint angle to a specific value with a defined velocity. The state s represents the agent's observation, which may include, for example, a sequence of recent camera images (e.g., the last four frames) as well as additional sensor inputs such as proprioceptive signals.

Mathematical description of the RL problem:

The RL modularities are represented as follows:

- Action: a
- Reward: r
- State: s
- Observation: o
- Policy: π
- Time step: t, e.g. a_t is the action taken at time step t, a_{t+1} is the action taken at the next time step, etc.
- s': The next state after taking action a in state s, i.e. $s' = s_{t+1}$, $s = s_t$.

1.6.1 Markov Decision Process

RL is based on the Markov decision process (MDP). A MDP is a stochastic decision-making process that formally models the transition probabilities from a state s to a subsequent state s' provided that action a is performed, expressed mathematically as $P(s'|s, a)$ [1, 7].

The objective is to learn a policy $\pi(a|s)$ that maximizes the expected long-term cumulative reward, defined as $\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$. The discount factor $\gamma \in [0, 1]$ determines the importance of future rewards compared to immediate rewards. A value of γ close to 0 makes the agent focus more on immediate rewards, while a value close to 1 encourages the agent to consider long-term rewards more heavily [1, 8].

A MDP also defines this reward function that specifies the reward r an agent receives when taking action a in state s and transitioning to a subsequent state s'. This is commonly expressed as $R_a(s, s')$ [1, 7].

The **Markov property** says that all information from the past is contained in the current state, mathematically expressed as $P(s_{t+1}|s_t) = P(s_{t+1}|s_1, s_2, ...s_t)$. In other words, the

transition probabilities depend only on the current state and not on the history of previous states [8].

In practice, the Markov property is often not strictly satisfied. For example, a single image captured by a camera does not provide sufficient information to infer the dynamics of the environment. From one image alone, it is not possible to determine whether a car is moving forward at 100 km/h or 10 km/h, reversing, or remaining stationary. However, such information is crucial for making correct decisions in applications like autonomous driving [9]. Therefore, in practice, only sufficiently informative state representations are often used because the ideal Markov property is not strictly fulfilled. This is often achieved by incorporating a history of observations, such as a sequence of the last five camera frames, to better capture the underlying dynamics of the environment [7].

1.6.2 Training Process

At the beginning of the training process, the policy is typically initialized randomly, meaning that the robot initially performs random movements. Through interaction with the environment, the agent explores different actions and attempts to identify those that yield high rewards. Actions associated with higher rewards are subsequently assigned a higher probability by the policy, making the robot more likely to select them in similar situations in the future.

A successful learning process requires a suitable balance between exploration and exploitation. Exploration refers to the evaluation of previously untried or infrequently selected actions in a given state, whereas exploitation denotes the repeated selection of actions that have already been identified as beneficial. During the early stages of training, greater emphasis is placed on exploration, as the agent must first acquire knowledge about the environment and determine which actions are most effective for accomplishing the task. As training progresses and the agent gains a better understanding of the environment and its reward structure, exploitation becomes increasingly important. This allows the agent to more frequently select actions that have proven successful, thereby improving overall task performance.

Numerous methods and algorithms have been proposed to effectively balance exploration and exploitation. Common examples include the ϵ-greedy strategy, Upper Confidence Bound (UCB), and Thompson Sampling. However, a detailed discussion of these methods is beyond the scope of this thesis.

Due to the exploration phase, the success of the chosen RL approach only becomes visible at a late stage, unlike in classical deep learning, see Figure 1.16.

1.6.3 Knowledge Modality from Video

An RL Knowledge Modality (KM) is some function learned from data that represents specific types of RL-related knowledge. KMs can be learned through trial-and-error principles or from various data sources, like video datasets, and can capture different aspects of the RL problem [10].

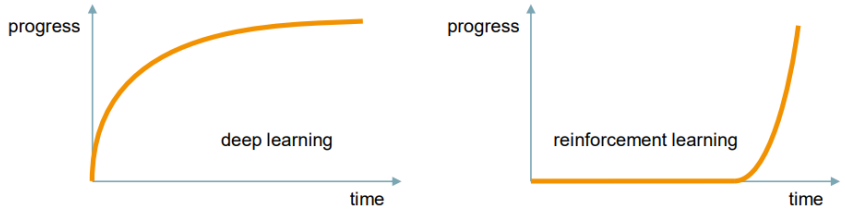


Figure 1.16: Training Progress in Deep Learning and Reinforcement Learning

KM from Video:

- *Policy:* A Policy $\pi(a_t|s_t)$ is a mapping from states to actions and defines the behavior of the agent that is in a particular state [1]. $\pi(a_t|s_t)$ is the probability of taking action a_t given that the agent is in state s_t. t denotes the time step. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. It can be learned from video data by observing the actions taken in different states and learning to predict those actions based on the visual input [10].
- *Value functions:* Value functions estimate the expected future return (cumulative reward) for a given state or state-action pair [1]. They can be learned from video data by observing the outcomes of actions and learning to predict the expected future rewards based on the visual input [10].
  - *State-value function (V-function):* A V-function $V_\pi(s_t)$ is a mapping from states to the expected future return when acting under a particular policy π [1].

    Intuition: "How good is it to be a certain state, if I follow my policy π?"

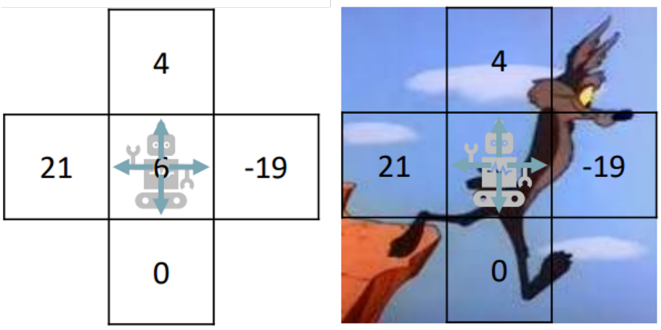


Figure 1.17: State-Value Function

  - *State-action value function (Q-function):* A Q-function $Q_\pi(s_t, a_t)$ is a mapping from state-action pairs to the expected future return when acting under a particular policy π [1].

    Intuition: "How good is this specific action here, if I continue behaving like my policy π?"
  - *V-function vs. Q-function:* The value function $V_\pi(s)$ estimates the expected return of being in a particular state s while following the policy π [1]. However, it does not provide information about the quality of individual

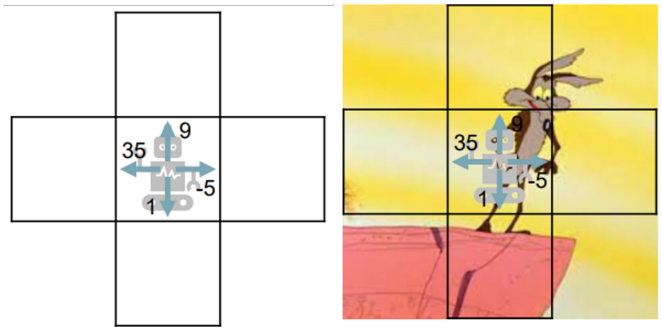


Figure 1.18: State-Action Value Function

actions available in that state s. In contrast, the State-action value function $Q_\pi(s, a)$ evaluates the expected return of taking a specific action a in state s and subsequently following the policy π [1]. Therefore, the Q-function offers a more detailed assessment by considering both the state and the chosen action. Figures 1.17 and 1.18 illustrate the concepts of the state-value function and the action-value function, respectively.

- *Dynamics model:* A Dynamics model $p(s_{t+1}|s_t, a_t)$ predicts the next state s_{t+1} given the current state s_t and action a_t [1]. Video prediction models can be used as robot dynamics models [11]. By learning to predict future video frames based on current observations and actions, the model can capture the underlying dynamics of the environment. This allows the robot to anticipate the consequences of its actions and make informed decisions based on predicted future states.
- *Reward model:* A reward function $r(s_t, a_t)$ defines the reward received by the agent for taking action a_t in state s_t [1]. It can be learned from video data by observing the outcomes of actions and inferring the underlying reward structure that motivates those actions. For example, if a video shows a robot successfully picking up an object, the reward function can learn to assign a positive reward to that action in similar states.

1.7 Imitation Learning

Imitation Learning (IL) or Learning from Demonstration is a general term for methods in which the agent derives or learns its behavior from expert demonstrations [12].

IL makes it possible to use "in the wild" videos—such as those found in great variety on the internet, for example on platforms like YouTube — to train AI models [13].

The demonstration videos can show a robot, a human, or even a loose sequence of actions, both in the real world or in a simulation environment, performing a specific task. However, videos without a clearly defined objective or with unknown activity can also be used [13].

This allows models to be trained to capture the visual and semantic relationships, as well as the physical dynamics of the real world. This, in turn, is a key prerequisite for

the development of a generalist policy. IL can be divided into Behavior Cloning and Learning from Videos/Observation.

1.7.1 Behavior Cloning

Behavior Cloning (BC) is a fundamental approach within IL, where a policy is learned directly from **annotated expert demonstrations** [11]. The training data consists of state-action pairs (s_t, a_t), and the problem is formulated as a standard supervised learning task.

Conceptually, BC can be understood as a form of direct imitation ("direct copying") of expert behavior, without explicitly modeling the underlying system dynamics or task objectives. Due to its simplicity and scalability, BC is one of the most widely used methods in robot learning from datasets.

Purpose: Policy Learning

BC learns a policy by supervised learning on expert demonstrations:

$$\pi_\theta(a_t \mid s_t) \tag{1.6}$$

where:

- s_t: State / Observation (images, proprioception, etc.) at time t
- a_t: Expert action at time t

The Policy $\pi_\theta(a \mid s)$ is optimized by minimizing the prediction error between the actions generated by the model and the corresponding expert actions.

$$\min_\theta \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\mathcal{L}(\pi_\theta(s), a)\right] \tag{1.7}$$

where:

- $\pi_\theta(s)$: The action your robot's policy predicts given state s.
- a: The actual action the expert took in that same state.
- $\mathcal{L}$: Usually Mean Squared Error (MSE) for continuous control or Cross-Entropy for discrete actions.
- $\mathbb{E}_{(s,a)\sim\mathcal{D}}[\cdot]$: Take all the state-action pairs (s, a) inside the expert dataset $\mathcal{D}$, and calculate the average value of the expression inside the brackets.

1.7.2 Learning from Videos / Observation

Learning from Videos (LfV), also referred to as Learning from Observation (LfO), is an IL approach that relies solely on **video demonstrations without explicit action annotations** [13, 14]. In contrast to BC, where state-action pairs are directly available, LfV aims to infer the underlying actions or high-level intent purely from perceptual

data, i.e. mapping observations to actions $(s \rightarrow a)$. It often uses **Representation Learning**, to learn and discover the most important (unknown) underlying data characteristics by transforming and reducing input in a fully data-driven and automatic manner.

A central challenge in LfV is the **inference problem**: determining which actions correspond to the observed behavior in the video [13]. Since no action labels are provided, additional methods and techniques are required to recover or approximate the missing action information. See chapter **XXX** for more information.

Due to this property, LfV can be seen as a **weakly supervised or self-supervised Imitation Learning** approach. Its main advantage lies in scalability, as it enables leveraging large amounts of unannotated video data, including internet videos or human demonstrations, without the need for costly data annotation [13]. Furthermore, additional actions can be derived from annotated video data too [13].

In robotics, LfV is commonly used as a **pretraining step**. The learned representations capture visual, semantic, and dynamical aspects of the environment, which can later be refined using methods such as BC with annotated data or in RL [13]. This combination allows models to first learn the real worlds dynamics from large-scale video data and subsequently specialize on task-specific robot demonstrations.

Purpose: Inferring Actions

LfV learns to infer actions from observations in a weakly supervised or self-supervised manner.

$$a_t = f(s_t, s_{t+1}) \tag{1.8}$$

where:

- a_t: The "latent" or predicted action.
- s_t: The current state/video frame.
- s_{t+1}: The next state/video frame.
- f: The inverse dynamics function (often a neural network) that maps state transitions to actions.

Bibliography

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] A. Vaswani *et al.*, "Attention is all you need," in *Neural Information Processing Systems*, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:13756489.

[3] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *ArXiv*, vol. abs/2010.11929, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:225039882.

[4] M. ud Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, "Vision language action models in robotic manipulation: A systematic review," *ArXiv*, vol. abs/2507.10672, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:280252663.

[5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:57246310.

[6] Walkerastro. "Action Space, State Space, Observation Space demystified", Medium. (08/26/2024), [Online]. Available: https://medium.com/@walkerastro41/action-space-state-space-observation-space-demystified-6c9c00a355b4 (visited on 04/28/2026).

[7] T. Miller. "Markov Decision Processes — Mastering Reinforcement Learning." (), [Online]. Available: https://gibberblot.github.io/rl-notes/single-agent/MDPs.html (visited on 04/28/2026).

[8] R. Jagtap. "Understanding the Markov Decision Process (MDP)", Built In. (), [Online]. Available: https://builtin.com/machine-learning/markov-decision-process (visited on 04/28/2026).

[9] C. Staff. "What Is a Markov Decision Process?", Coursera. (15. 05. 2025), Adresse: https://www.coursera.org/articles/what-is-a-markov-decision-process (besucht am 28. 04. 2026).

[10] M. Wulfmeier, A. Byravan, S. Bechtle, K. Hausman, and N. M. O. Heess, "Foundations for transfer in reinforcement learning: A taxonomy of knowledge modalities," *ArXiv*, vol. abs/2312.01939, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:265609417.

[11] C. Eze and C. Crick, "Learning by watching: A review of video-based learning approaches for robot manipulation," *IEEE access : practical innovations, open solutions*, vol. 13, pp. 184 071–184 109, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:267627541.

[12] I. Chrysomallis and G. Chalkiadakis, "Imitation learning in the deep learning era: A novel taxonomy and recent advances," *ArXiv*, vol. abs/2511.03565, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:282758415.

[13] R. McCarthy *et al.*, "Towards generalist robot learning from internet video: A survey," *Journal of Artificial Intelligence Research*, vol. 83, 07/2025, ISSN: 1076-9757. DOI: 10.1613/jair.1.17400. [Online]. Available: http://dx.doi.org/10.1613/jair.1.17400.

[14] R. Burnwal, H. Mehta, N. P. Bhatt, and B. Ravindran, "Learning from observation: A survey of recent advances," *ArXiv*, vol. abs/2509.19379, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:281505412.

List of Figures

List of Tables